]

STRUCTURED ANALYSIS & STRUCTURED DESIGN

Hotel Automation Software

Document Type	Structured Analysis & Structured Design (SA/SD)
Subject	Software Engineering
Subject Code	CS3004
Version	1.0 (PlantUML-rendered diagrams)
Date	April 22, 2026
Based on SRS	SRS_HotelAutomation_v1.0
Group Members	Suyash Pandey : 2023UG1100 Suraj Kumar : 2023UG1106 Himanshu Raj : 2023UG1092 Satyendra Kumar : 2023UG1105
Institution	Indian Institute of Information Technology Ranchi, Jharkhand

Copying of assignments is considered a serious offense.

Contents

1 Introduction **3**
1.1 Purpose . . . 3
1.2 Notation . . . 3
1.3 Diagram Toolchain . . . 3
1.4 Traceability . . . 3

2 Structured Analysis **4**
2.1 Context Diagram (Level 0) . . . 4
2.2 Level-1 DFD . . . 6
2.2.1 Process Summaries . . . 6
2.2.2 Data Store Summaries . . . 7
2.3 Level-2 DFD — Explosion of 2.0 Process Reservation . . . 8
2.3.1 Process Summaries (Level 2) . . . 8
2.4 Data Dictionary . . . 10

3 Structured Design **12**
3.1 Design Strategy . . . 12
3.2 Structure Chart . . . 12
3.2.1 Data and Flag Couples . . . 12
3.3 Module Specifications . . . 14

4 Requirements Traceability **17**

5 Revision History **18**

1 Introduction

1.1 Purpose

This document presents the Structured Analysis (SA) and Structured Design (SD) for the **Hotel Automation Software**. It decomposes the system defined in the Software Requirements Specification (SRS v1.0) into data-flow diagrams (DFDs), a data dictionary, a structure chart, and module specifications suitable for direct implementation in Java / Java Swing backed by an embedded relational database.

1.2 Notation

DFD notation:

- **Process** — blue ellipse, labelled `n.m Name`.
- **External entity** — orange rectangle.
- **Data store** — green cylinder, labelled `Dn Name`.
- **Data flow** — directed arrow with a data-name label.

(PlantUML renders data stores as cylinders rather than the classical open-rectangle; the semantics are identical.)

Structure-chart notation:

- **Module** — rectangle labelled with the module name.
- **Call** — arrow from caller to callee.
- Data and flag couples are listed in Table 3.2.1 rather than on the diagram itself, to keep the layout readable.

1.3 Diagram Toolchain

Diagrams are authored in **PlantUML** (source in `puml/`) and rendered to PNG via the PlantUML online editor or a local `plantuml.jar`. The PNGs live in `images/` and are embedded below via `\includegraphics`. See `SA_SD/README.md` for rendering instructions.

1.4 Traceability

Every process in the DFDs and every module in the structure chart is cross-referenced to the SRS requirements it realises. A consolidated traceability matrix appears in Section 4.

2 Structured Analysis

2.1 Context Diagram (Level 0)

The Level-0 (context) DFD treats the entire system as a single process and shows only its interactions with external entities.

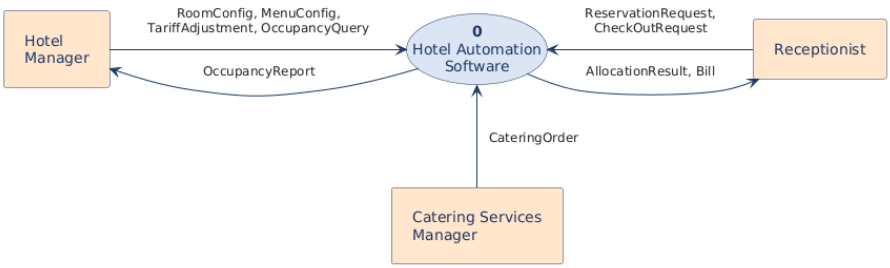

Figure 1: Context Diagram (Level 0)

External entities. The system interacts with four distinct external entities corresponding to the user classes defined in SRS §2.3:

- **Hotel Manager** — configures rooms, tariffs, food menu, and discount policies; reviews occupancy reports.
- **Receptionist** — manages guest reservations, room allocations, token issuance, and check-out.
- **Catering Services Manager** — records food and beverage orders against guest token numbers.
- **Guest** — the indirect beneficiary; interacts via the Receptionist and Catering Services Manager.

Data flows.

- **RoomConfig** — room numbers, bed types, AC types, and initial tariffs supplied by the Hotel Manager.
- **TariffAdjustment** — room type and percentage adjustment value supplied by the Hotel Manager.
- **OccupancyQuery** — month selection from the Hotel Manager; system returns **OccupancyReport**.
- **MenuConfig** — food item names and unit prices supplied by the Hotel Manager.
- **ReservationRequest** — guest name, arrival date/time, duration, room type preference, advance payment, optional frequent-guest identity number, supplied by the Receptionist.
- **AllocationResult** — room number and token number (success) or apology message (failure) returned to the Receptionist.

- **CateringOrder** — token number, food item, quantity, date and time supplied by the Catering Services Manager.
- **CheckOutRequest** — token number supplied by the Receptionist to initiate billing.
- **Bill** — itemised bill with net balance payable, returned to the Receptionist for printing.

2.2 Level-1 DFD

Process 0 is decomposed into six cooperating processes and six data stores.

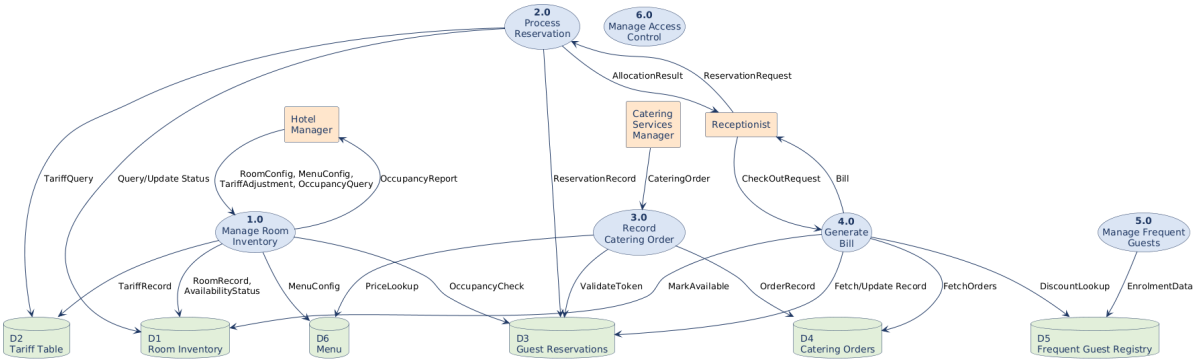

Figure 2: Level-1 DFD — six cooperating processes and six data stores.

2.2.1 Process Summaries

ID	Process	Purpose	SRS Ref
1.0	Manage Room Inventory	Accept room configuration and menu inputs from the Hotel Manager; validate and persist room records, tariffs, and food-item prices; compute and return the occupancy report for a given month; apply tariff revision requests.	FR-1.*, FR-2.*, FR-5.3, FR-8.7, FR-8.8
2.0	Process Reservation	Accept reservation requests from the Receptionist; check room availability for the requested type and dates; allocate a room and issue a token number, or generate an apology message if no suitable room is available. Exploded in §2.3.	FR-3.*, FR-4.*, FR-8.2–8.4
3.0	Record Catering Order	Accept catering orders from the Catering Services Manager; validate the token number against active guest records; persist the order against the guest's account for later billing.	FR-5.1–5.4

ID	Process	Purpose	SRS Ref
4.0	Generate Bill	On check-out request, retrieve the guest's room charge and accumulated catering charges; apply advance payment and any frequent-guest discount; produce and return an itemised bill; mark the room as available.	FR-6.1–6.6, FR-8.5–8.6
5.0	Manage Frequent Guests	Maintain the frequent-guest registry; validate loyalty identity numbers at reservation time; enrol new frequent guests; supply the applicable discount percentage to the billing process.	FR-7.1–7.4
6.0	Manage Access Control	Enforce role-based access: restrict panel visibility and operations to authorised user classes (Hotel Manager, Receptionist, Catering Services Manager).	NFR-S1, FR-8.1

2.2.2 Data Store Summaries

ID	Data Store	Contents
D1	Room Inventory	Per-room records: room number, bed type, AC type, current availability status, and the tariff category applicable at check-in.
D2	Tariff Table	Current and historical tariffs per room type (single-AC, single-Non-AC, double-AC, double-Non-AC), including the tariff change log (room type, old tariff, new tariff, date of change).
D3	Guest Reservations	Per-reservation records: token number, guest name, room number, arrival date/time, approximate and actual check-out date/time, advance payment, and optional frequent-guest identity number.
D4	Catering Orders	Per-order records: token number, food item name, quantity, unit price at time of order, date and time of consumption.
D5	Frequent Guest Registry	Loyalty records: identity number, guest name, contact information, and the configured discount percentage.
D6	Menu	Food and beverage items with current unit prices, configurable by the Hotel Manager.

2.3 Level-2 DFD — Explosion of 2.0 Process Reservation

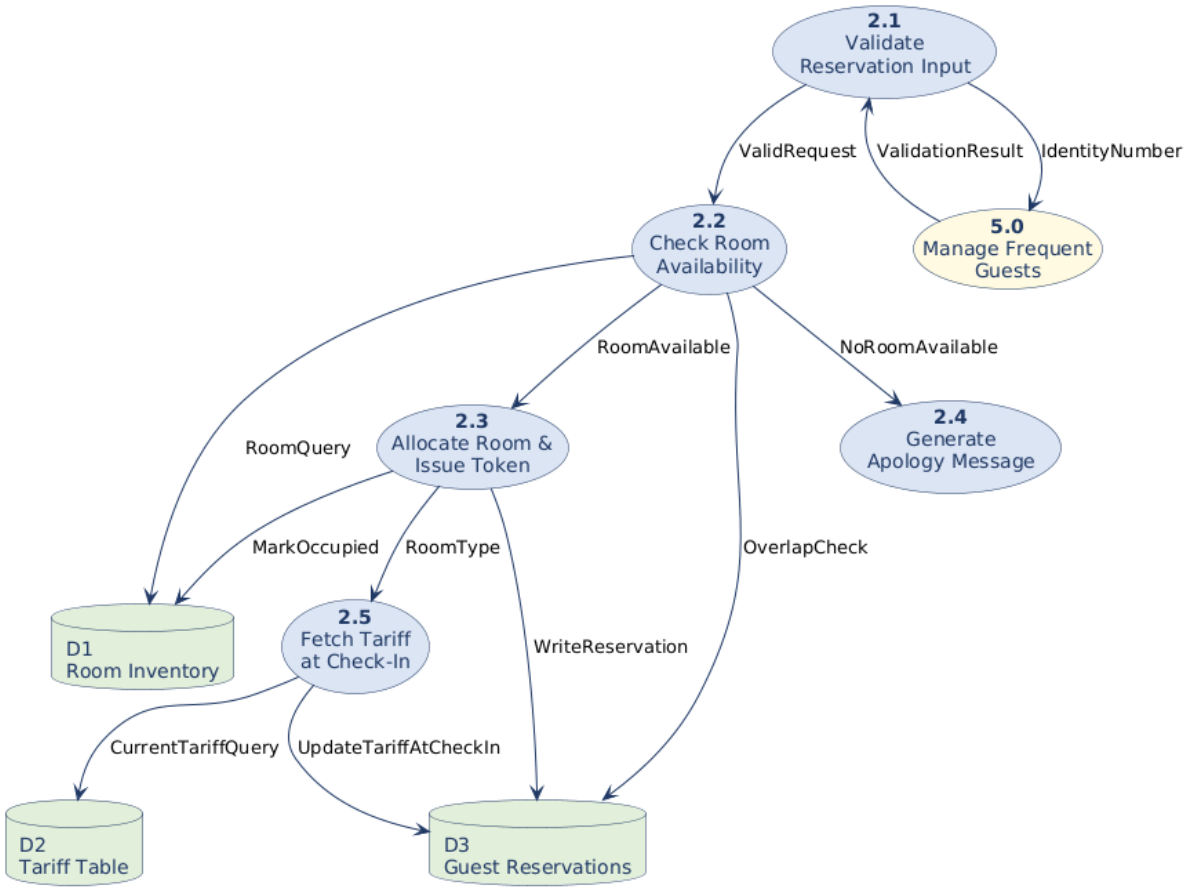

Figure 3: Level-2 DFD: explosion of **2.0 Process Reservation**. 5.0 Manage Frequent Guests (shaded) is an upstream process from the Level-1 DFD consulted during loyalty validation.

2.3.1 Process Summaries (Level 2)

ID	Process	Purpose (SRS ref in brackets)
2.1	Validate Reservation Input	Accept raw reservation data from the Receptionist; check that all mandatory fields are non-empty and within valid ranges (arrival date not in the past, duration $\geq$ 1 night, advance payment $\geq$ 0); if a frequent-guest identity number is supplied, delegate validation to 5.0. [FR-3.1, FR-3.2, FR-8.9]
2.2	Check Room Availability	Query D1 to find rooms matching the requested bed type and AC type that are not occupied for the requested date range; return the first available room number (lowest room number), or a `NoRoomAvailable` flag. [FR-4.1, NFR-R3]

ID	Process	Purpose (SRS ref in brackets)
2.3	Allocate Room & Issue Token	If a room is available, mark it `occupied` in D1 for the reservation period; generate a unique token number; write the reservation record to D3; return the room number and token number to the Receptionist. [FR-4.2, FR-4.4]
2.4	Generate Apology Message	If no suitable room is available, compose an apology message stating the unavailability and the reason (no room of the requested type on the requested dates) and return it to the Receptionist. [FR-4.3]
2.5	Fetch Tariff at Check-In	Retrieve the current tariff for the allocated room type from D2 and record it in the reservation entry in D3, so that billing uses the tariff prevailing at check-in time rather than the tariff at check-out time. [FR-6.2]

2.4 Data Dictionary

The data dictionary uses the standard structured-analysis notation: `=` (is composed of), `+` (and), `[ | ]` (either-or), `{ }` (iteration), `( )` (optional), `**` (comment).

Name	Definition
`RoomConfig`	= {RoomRecord} `RoomRecord` = RoomNumber + BedType + ACType `BedType` = [`Single` | `Double`] `ACType` = [`AC` | `Non-AC`]
`TariffAdjustment`	= RoomType + AdjustmentPct ** −100 < AdjustmentPct; positive = upward revision **
`OccupancyQuery`	= MonthYear ** format YYYY-MM **
`OccupancyReport`	= MonthYear + TotalRoomNightsAvailable + TotalRoomNightsOccupied + AverageOccupancyPct
`MenuConfig`	= {FoodItemRecord} `FoodItemRecord` = FoodItemName + UnitPrice
`ReservationRequest`	= GuestName + ArrivalDateTime + ApproxDurationNights + RoomTypePreference + AdvancePaid + (IdentityNumber) `RoomTypePreference` = BedType + ACType
`AllocationResult`	= [SuccessResult | ApologyMessage] `SuccessResult` = RoomNumber + TokenNumber `ApologyMessage` = WarningText + UnavailableRoomType + RequestedDates
`CateringOrder`	= TokenNumber + FoodItemName + Quantity + OrderDateTime
`CheckOutRequest`	= TokenNumber
`Bill`	= GuestName + RoomNumber + TokenNumber + CheckInDateTime + CheckOutDateTime + {RoomChargeLineItem} + {CateringLineItem} + TotalBillAmount + AdvancePaidDeducted + (DiscountDeducted) + NetBalancePayable `RoomChargeLineItem` = TariffPerNight + ActualNightsStayed + RoomChargeSubtotal `CateringLineItem` = FoodItemName + Quantity + UnitPrice + CateringSubtotal
`RoomRecord`	= RoomNumber + BedType + ACType + [`available` | `occupied`]
`TariffRecord`	= RoomType + CurrentTariff + {TariffChangeEntry} `TariffChangeEntry` = OldTariff + NewTariff + ChangeDate
`ReservationRecord`	= TokenNumber + GuestName + RoomNumber + RoomType + TariffAtCheckIn + ArrivalDateTime + ApproxDurationNights + (ActualCheckOutDateTime) + AdvancePaid + (IdentityNumber)

Name	Definition
`CateringOrderRecord`	= TokenNumber + FoodItemName + Quantity + UnitPrice + OrderDateTime
`FrequentGuestRecord`	= IdentityNumber + GuestName + ContactInfo + DiscountPct
`TokenNumber`	= Integer ** system-generated, unique across lifetime of DB **

3 Structured Design

3.1 Design Strategy

The Level-1 DFD exhibits a classical **transaction-centred** shape: afferent flows (`ReservationRequest`, `CateringOrder`, `CheckOutRequest`, `TariffAdjustment`, `OccupancyQuery`) arrive at a transaction centre which dispatches each transaction to one of several dedicated action branches (Reservation, Catering, Billing, Room Management, Frequent-Guest Management). We therefore apply **transaction analysis** to derive the structure chart: one root coordinator with a transaction dispatcher and one sub-tree per transaction type, plus shared utility modules for database access and input validation.

3.2 Structure Chart

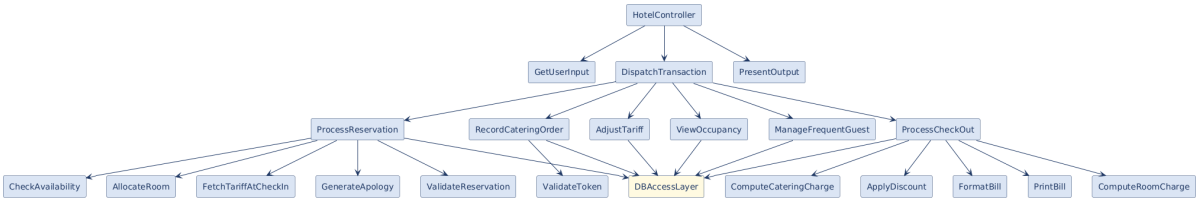

Figure 4: Structure Chart (transaction-analysis derived). `DBAccessLayer` (shaded) is a shared utility module invoked across all transaction branches. Data / flag couples are listed in Table 3.2.1.

3.2.1 Data and Flag Couples

The following couples annotate the calls in Figure 4. "Down" = caller → callee, "Up" = callee → caller.

Caller	Callee	Couple	Payload
HotelController	GetUserInput	data (up)	TransactionRequest
HotelController	Dispatch-Transaction	data (down)	TransactionRequest
HotelController	PresentOutput	data (down)	TransactionResult
Dispatch-Transaction	Process-Reservation	data (down) data (up)	ReservationRequest AllocationResult
Dispatch-Transaction	Record-CateringOrder	data (down) flag (up)	CateringOrder OrderOkFlag
Dispatch-Transaction	ProcessCheckOut	data (down) data (up)	CheckOutRequest Bill
Dispatch-Transaction	AdjustTariff	data (down) flag (up)	TariffAdjustment AdjustOkFlag
Dispatch-Transaction	ViewOccupancy	data (down) data (up)	OccupancyQuery OccupancyReport

Caller	Callee	Couple	Payload
Process-Reservation	Validate-Reservation	data (down) flag (up)	ReservationRequest ValidFlag, ErrorList
Process-Reservation	CheckAvailability	data (down) data (up)	RoomTypePreference, DateRange RoomNumber or `null`
Process-Reservation	AllocateRoom	data (down) data (up)	ReservationRequest, RoomNumber TokenNumber
Process-Reservation	GenerateApology	data (down) data (up)	RoomTypePreference, DateRange ApologyMessage
ProcessCheckOut	Compute-RoomCharge	data (down) data (up)	ReservationRecord RoomChargeSubtotal
ProcessCheckOut	Compute-CateringCharge	data (down) data (up)	TokenNumber CateringChargeSubtotal
ProcessCheckOut	ApplyDiscount	data (down) data (up)	IdentityNumber, TotalCharge DiscountAmt
ProcessCheckOut	FormatBill	data (down) data (up)	BillComponents Bill
Record-CateringOrder	ValidateToken	data (down) flag (up)	TokenNumber ActiveFlag
$*$ (all branches)	DBAccessLayer	data (down) data (up)	Query / WriteRequest ResultSet or `null`

Coupling / cohesion.

- All data couples are *simple data* (no tramp data); no global state is shared between modules other than through the database access layer.
- `ValidFlag`, `OrderOkFlag`, `AdjustOkFlag`, and `ActiveFlag` are control couples; each encodes a single binary outcome and justifies its role as a genuine control signal.
- Every leaf module exhibits **functional cohesion** (one well-defined job).
- The `Process-Reservation` and `ProcessCheckOut` modules are **sequential cohesive**: each step feeds the next in a well-defined pipeline.
- `DBAccessLayer` is a **communicational cohesion** shared-service module whose sole responsibility is mediated database I/O, keeping all other modules database-agnostic.

3.3 Module Specifications

Each module is specified as **(Inputs | Outputs | Processing)**.

Module	Specification (SRS ref)
`HotelController`	**In:** GUI events from all user classes. **Out:** lifecycle transitions, displayed results. **Proc:** Top-level coordinator. Authenticates the active user class, enables or disables panels per role (NFR-S1). Sequences `GetUserInput` → `Dispatch-Transaction` → `PresentOutput` for each user action. Runs background database tasks on a worker thread to preserve GUI responsiveness (NFR-P3). [FR-8.1, NFR-S1]
`GetUserInput`	**In:** Swing form field values and button events. **Out:** `TransactionRequest` (typed DTO). **Proc:** Pulls values from the active form panel into a typed DTO; determines the transaction type (reservation, catering, check-out, tariff adjustment, occupancy query, frequent-guest enrolment). [FR-8.2–8.10]
`Dispatch-Transaction`	**In:** `TransactionRequest`. **Out:** `TransactionResult`. **Proc:** `switch` on transaction type; delegates to `Process-Reservation`, `Record-CateringOrder`, `ProcessCheckOut`, `AdjustTariff`, `ViewOccupancy`, or `ManageFrequentGuest`.
`PresentOutput`	**In:** `TransactionResult`. **Out:** rendered GUI panel / dialog. **Proc:** Formats and displays the result in the appropriate output panel; invokes `PrintBill` if the result is a `Bill` and the user confirms printing. [FR-8.5, FR-8.6]
`Process-Reservation`	**In:** `ReservationRequest`. **Out:** `AllocationResult`. **Proc:** Orchestrates `Validate-Reservation` → `CheckAvailability`; if available, calls `AllocateRoom` and `FetchTariffAtCheckIn`; otherwise calls `GenerateApology`. [FR-3.*, FR-4.*]
`Validate-Reservation`	**In:** `ReservationRequest`. **Out:** `ValidFlag`, `ErrorList`. **Proc:** Checks mandatory fields, date validity, duration ≥ 1, advance payment ≥ 0; if `IdentityNumber` present, calls `LookupFrequentGuest` to confirm it exists. [FR-3.1, FR-8.9]
`Check-Availability`	**In:** `RoomTypePreference`, `DateRange`. **Out:** lowest available `RoomNumber` or `null`. **Proc:** Queries D1 via `DBAccessLayer` for rooms of the requested type; filters out rooms with overlapping active reservations from D3; returns the lowest-numbered match. [FR-4.1, NFR-R3]

Module	Specification (SRS ref)
`AllocateRoom`	**In:** `ReservationRequest`, `RoomNumber`. **Out:** `TokenNumber`. **Proc:** Generates a unique `TokenNumber` (auto-increment primary key); writes the reservation record to D3 via `DBAccessLayer`; marks the room `occupied` in D1 for the reservation period. [FR-4.2, FR-4.4]
`FetchTariff-AtCheckIn`	**In:** `RoomType`. **Out:** `TariffAtCheckIn`. **Proc:** Reads the current tariff for the room type from D2 via `DBAccessLayer` and stores it in the reservation record so that billing is insulated from future tariff changes. [FR-6.2]
`GenerateApology`	**In:** `RoomTypePreference`, `DateRange`. **Out:** `ApologyMessage`. **Proc:** Composes a message stating the requested type and dates for which no room is available. [FR-4.3]
`Record-CateringOrder`	**In:** `CateringOrder`. **Out:** `OrderOkFlag`. **Proc:** Calls `ValidateToken`; if valid, looks up the unit price of the food item from D6 via `DBAccessLayer`; writes the order record to D4. [FR-5.1–5.4]
`ValidateToken`	**In:** `TokenNumber`. **Out:** `ActiveFlag`. **Proc:** Queries D3 via `DBAccessLayer` to confirm the token corresponds to a currently checked-in guest (arrival $\leq$ now and no `ActualCheckOutDateTime` recorded). [FR-5.2]
`ProcessCheckOut`	**In:** `CheckOutRequest` (token number). **Out:** `Bill`. **Proc:** Retrieves reservation record from D3; calls `ComputeRoom-Charge`, `ComputeCater-Charge`, and optionally `ApplyDiscount`; calls `FormatBill`; records actual check-out date/time in D3; marks room `available` in D1. [FR-6.1–6.6]
`Compute-RoomCharge`	**In:** `ReservationRecord`. **Out:** `RoomChargeSubtotal`. **Proc:** $\text{RoomCharge} = \text{TariffAtCheckIn} \times \lceil(\text{CheckOut} - \text{CheckIn})/24\text{h}\rceil$. Rounds to two decimal places. [FR-6.2, NFR-R4]
`Compute-CateringCharge`	**In:** `TokenNumber`. **Out:** {CateringLineItem}, `CateringChargeSubtotal`. **Proc:** Retrieves all catering orders for the token from D4; $\text{CateringCharge} = \sum(\text{Quantity}_i \times \text{UnitPrice}_i)$. Rounds to two decimal places. [FR-6.3, NFR-R4]
`ApplyDiscount`	**In:** `IdentityNumber`, `TotalCharge`. **Out:** `DiscountAmt`. **Proc:** Looks up `DiscountPct` for the identity number in D5 via `DBAccessLayer`; $\text{DiscountAmt} = \text{TotalCharge} \times \text{DiscountPct}/100$. [FR-7.3, NFR-R4]

Module	Specification (SRS ref)
`FormatBill`	**In:** `BillComponents` (reservation record, line items, charges, discount, advance paid). **Out:** `Bill`. **Proc:** Assembles the itemised bill DTO per FR-6.4; NetBalance = TotalCharge − DiscountAmt − AdvancePaid. Two-decimal rounding throughout. [FR-6.4, NFR-R4]
`PrintBill`	**In:** `Bill`. **Out:** print job. **Proc:** Formats the bill as a printable layout and submits to the Java `PrinterJob` API targeting the system's default printer. [FR-6.5]
`AdjustTariff`	**In:** `TariffAdjustment`. **Out:** `AdjustOkFlag`. **Proc:** Reads current tariff from D2; computes NewTariff = CurrentTariff × (1 + AdjPct/100); writes new tariff and appends a `TariffChangeEntry` to D2 via `DBAccessLayer`. [FR-2.2, FR-2.3]
`ViewOccupancy`	**In:** `OccupancyQuery`. **Out:** `OccupancyReport`. **Proc:** Queries D3 for all reservations overlapping the selected month; computes `TotalRoomNightsOccupied` and `TotalRoomNightsAvailable` from D1; calculates `AverageOccupancyPct`. [FR-2.1]
`Manage-FrequentGuest`	**In:** enrolment data or lookup request. **Out:** `IdentityNumber` (enrolment) or `FrequentGuestRecord` (lookup). **Proc:** On enrolment, generates a new `IdentityNumber` and writes to D5; on lookup, retrieves record from D5 by identity number and returns the discount percentage. [FR-7.1, FR-7.2, FR-7.4]
`Lookup-FrequentGuest`	**In:** `IdentityNumber`. **Out:** `ValidFlag`, `DiscountPct`. **Proc:** Queries D5 via `DBAccessLayer`; returns `true` and discount if found, `false` otherwise. [FR-7.3]
`DBAccessLayer`	**In:** typed query or write request. **Out:** `ResultSet` or `null`. **Proc:** Single-responsibility database gateway. Translates typed DTOs into parameterised SQL statements; executes against the embedded RDBMS; maps result rows back to typed objects. All transactional commits are performed here. No business logic resides in this module. [NFR-R5]

4 Requirements Traceability

SRS Req. ID	DFD Process	Data Store	Structure-chart Module
FR-1.1 – FR-1.3	1.0	D1	DBAccessLayer (room writes)
FR-2.1	1.0, 6.0	D1, D3	ViewOccupancy
FR-2.2 – FR-2.3	1.0	D2	AdjustTariff
FR-3.1 – FR-3.2	2.0, 2.1	—	Validate-Reservation
FR-4.1	2.2	D1, D3	CheckAvailability
FR-4.2, FR-4.4	2.3	D1, D3	AllocateRoom
FR-4.3	2.4	—	GenerateApology
FR-5.1 – FR-5.4	3.0	D4, D6	Record-CateringOrder, ValidateToken
FR-5.3	1.0	D6	DBAccessLayer (menu writes)
FR-6.1 – FR-6.4	4.0	D3, D4	ProcessCheckOut, ComputeRoom-Charge, ComputeCater-Charge, FormatBill
FR-6.2	2.5	D2, D3	FetchTariffAtCheckIn
FR-6.5	4.0	—	PrintBill
FR-6.6	4.0	D1	ProcessCheckOut (room release)
FR-7.1 – FR-7.2	5.0	D5	ManageFrequentGuest
FR-7.3	5.0, 4.0	D5	LookupFrequentGuest, ApplyDiscount
FR-7.4	5.0	D5	ManageFrequentGuest (enrolment)
FR-8.1 – FR-8.10	1.0, 2.0, 4.0, 5.0, 6.0	—	HotelController, GetUserInput, PresentOutput
NFR-P3	—	—	HotelController (background thread)
NFR-R3	2.2	D1, D3	CheckAvailability (overlap check)
NFR-R4	4.0	—	ComputeRoom-Charge, ComputeCater-Charge, ApplyDiscount, FormatBill

SRS Req. ID	DFD Process	Data Store	Structure-chart Module
NFR-R5	—	D1–D6	DBAccessLayer (transactional commits)
NFR-S1	6.0	—	HotelController (role-based panel control)

5 Revision History

Version	Date	Author	Description
1.0	April 22, 2026	Project Team	Initial SA/SD for Hotel Automation Software. Context diagram, Level-1 DFD (six processes, six data stores), Level-2 explosion of Process Reservation, data dictionary, transaction-analysis-derived structure chart with module specifications, and full traceability matrix. Based on SRS_HotelAutomation.

End of Document — Hotel Automation Software SA/SD v1.0